

Universidad de Palermo
Maestría en Inteligencia Artificial
Redes Neuronales y Deep Learning

Memoria o patrones locales

RNN, LSTM y CNN 1D para clasificar movimiento

Trabajo práctico final

Profesor: Alan Dreszman

Alumno: Nicolás Ferrero

Dataset: UCI Human Activity Recognition Using Smartphones

Repositorio: <https://github.com/NicoFerrero/tp-redes-neuronales-deep-learning>

Fecha de entrega: 07/10/2026

1. Introducción	1
2. Datos	1
2.1 El dataset	1
2.2 Partición por persona	2
2.3 Normalización	2
3. Modelos y decisiones de diseño	2
3.1 Referencia sin redes: regresión logística	2
3.2 CNN 1D	2
3.3 RNN simple	3
3.4 El problema: el gradiente que se desvanece	3
3.5 LSTM	3
3.6 Híbrido CNN + LSTM	4
4. Entrenamiento y métricas	4
4.1 Función de pérdida: entropía cruzada	5
4.2 Recorte de gradiente	5
4.3 Métricas	5
5. Resultados	6
6. Análisis	7
6.1 ¿Los modelos usan el orden temporal?	7
6.2 Cuánto influye cada paso de la ventana	7
7. Lo que no funcionó y lo que aprendí	8
8. Qué haría con más tiempo	8
9. Conclusiones	9
Referencias	9
Anexo A. Ayuda visual de la matemática detrás de las CNN	10
A.1 Padding: por qué la señal no se acorta	10
A.2 Campo receptivo: cuánto ve cada neurona	10
A.3 De dónde sale el 0,58	11

1. Introducción

Este trabajo parte de una pregunta concreta: cuando los datos son una serie temporal, ¿conviene usar una red que recuerda lo que pasó antes? Las redes recurrentes leen la señal paso a paso y mantienen un estado que resume lo visto hasta ese momento. Una CNN 1D, en cambio, mira ventanas cortas de la señal y busca patrones locales. Mi hipótesis inicial era que la memoria de las recurrentes iba a ser una ventaja.

Para ponerla a prueba usé el dataset UCI HAR, con señales de acelerómetro y giroscopio de un celular, y comparé cuatro redes con la misma cantidad de parámetros: una CNN 1D, una RNN simple, una LSTM y un híbrido CNN + LSTM. Como referencia sin redes usé una regresión logística. El recorrido fue incremental. Primero comparé la CNN contra la RNN simple, después usé la LSTM para corregir el problema que mostró la RNN, y por último probé si juntar convolución y memoria mejoraba el resultado.

Además de medir el accuracy hice dos experimentos para entender qué usa cada red por dentro. En el primero desordené la señal por bloques para ver si los modelos dependen del orden. En el segundo medí cuánto influye cada paso de la ventana en el aprendizaje, para observar el desvanecimiento del gradiente que predice la teoría.

2. Datos

2.1 El dataset

UCI Human Activity Recognition Using Smartphones (Anguita y otros, 2013) tiene registros de 30 personas de entre 19 y 48 años que realizaron seis actividades con un celular Samsung Galaxy S II en la cintura: caminar, subir escaleras, bajar escaleras, estar sentado, estar parado y estar acostado. El celular registró a 50 Hz la aceleración y la velocidad de rotación en tres ejes. Los autores cortaron la señal en ventanas de 2,56 segundos (128 muestras) con 50% de solapamiento. El dataset es público y tiene licencia CC BY 4.0.

Cada ventana tiene 9 canales: la aceleración total (que incluye la gravedad), la aceleración del cuerpo (la misma señal sin la gravedad) y la velocidad de rotación del giroscopio, cada una en los ejes x, y, z. El dataset también trae 561 features calculadas a mano, pero decidí trabajar solo con la señal cruda, porque la pregunta del trabajo es qué puede aprender cada red a partir de la secuencia.

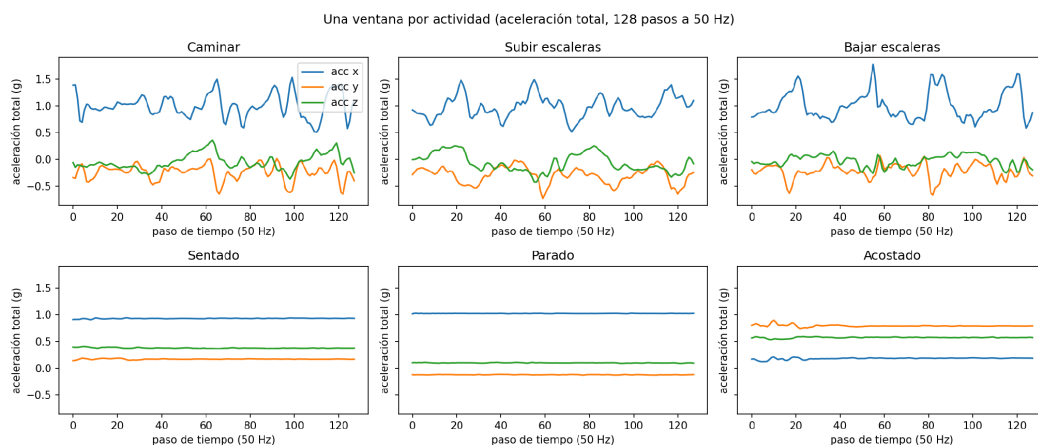


Figura 1. Una ventana de ejemplo por actividad (aceleración total en los tres ejes).

La figura 1 anticipa algo importante. Las tres actividades dinámicas tienen oscilaciones que se repiten a lo largo de la ventana, mientras que las tres estáticas son casi planas y se diferencian solo en la altura de las curvas, es decir, en cómo está orientado el celular respecto de la gravedad.

2.2 Partición por persona

El test oficial tiene 9 personas que no aparecen en train. De las 21 personas restantes separé 5 para validación. Así quedaron 16 personas para entrenar (5.619 ventanas), 5 para validar (1.733) y 9 para evaluar (2.947). Separar por persona y no por ventana fue una decisión importante por dos motivos. Lo que quiero saber es si el modelo reconoce la actividad de alguien que nunca vio. Además, dos ventanas seguidas comparten la mitad de la señal, así que una partición al azar habría puesto casi la misma ventana en train y en validación, y la validación habría dado un resultado engañoso.

2.3 Normalización

Los canales tienen escalas distintas. La aceleración total ronda 1 g por la gravedad y la del cuerpo está cerca de 0. Para que ninguno domine, a cada canal le resté su media y lo dividí por su desvío, calculados solo con train:

$$x_{\text{normalizado}} = (x - \text{media del canal}) / \text{desvío del canal}$$

Ejemplo: si un canal tiene media 0,8 y desvío 0,4, un valor de 1,2 pasa a $(1,2 - 0,8) / 0,4 = 1$. Eso se lee como "un desvío por encima de lo habitual para ese canal". Un valor de 0,8 pasa a 0, o sea, justo en el promedio.

Usé una sola media por canal y no una por paso de tiempo, porque una aceleración vale lo mismo al principio o al final de la ventana. Calcular la media y el desvío solo con train evita que se filtre información de validación o de test.

3. Modelos y decisiones de diseño

Los cuatro modelos tienen alrededor de 25 mil parámetros (figura 2). Igualar el tamaño fue deliberado. Si una red tuviera muchos más parámetros que otra, no podría saber si gana por su arquitectura o por ser más grande.

3.1 Referencia sin redes: regresión logística

Aplané cada ventana en un vector de $128 \times 9 = 1.152$ valores y entrené una regresión logística, que asigna un peso a cada valor y los combina para puntuar cada clase. El punto débil es que aprende un peso fijo para cada posición de la ventana. Si en entrenamiento un pico de aceleración aparece en el paso 10, el modelo aprende a buscarlo ahí. Si en una ventana nueva el mismo pico aparece en el paso 30, porque el corte cayó en otro momento, lo multiplica por otro peso y no lo reconoce. La usé como piso, para saber si las redes aprenden algo que un modelo lineal no puede.

3.2 CNN 1D

Una convolución con un filtro de 5 pasos que recorre la señal buscando un patrón, por ejemplo un pico de aceleración o una subida brusca. El mismo filtro se aplica en todas las posiciones, así que el patrón se detecta igual en cualquier lugar de la ventana. La primera capa tiene 32 filtros, cada uno especializado en un patrón distinto. Para que la señal no se acorte en los bordes agregué dos ceros a cada lado (padding), y así cada capa conserva los 128 pasos (anexo A.1).

Apilar capas amplía lo que ve cada neurona. Una neurona de la primera capa ve 5 pasos. Una de la segunda combina neuronas vecinas de la primera y ve 9. Después un MaxPool resume los pasos de a dos, y la tercera capa termina viendo 18 pasos, unos 0,36 segundos (anexo A.2). Es menos que un paso completo al caminar, que dura alrededor de medio segundo. Al final, un promedio global resume cada filtro en un número que indica cuánto apareció su patrón en toda la ventana, y una capa lineal convierte esos 64 números en las 6 clases.

3.3 RNN simple

La RNN lee la ventana paso a paso, como alguien que lee una oración palabra por palabra y va recordando lo anterior. En cada paso combina el dato nuevo con lo que venía recordando:

$$h_t = \tanh(W \cdot x_t + U \cdot h_{t-1} + b)$$

donde x_t son los 9 canales en el paso t , h_{t-1} es el estado que traía del paso anterior, y W y U son los pesos que aprende la red, que se usan iguales en los 128 pasos. La $\tanh$ mantiene cada valor entre -1 y 1 . El estado del último paso resume toda la ventana y pasa a una capa lineal que da las 6 clases. A diferencia de la CNN, una sola capa ya ve toda la ventana.

3.4 El problema: el gradiente que se desvanece

Para aprender, el error que se calcula al final tiene que viajar hacia atrás hasta los primeros pasos. En la RNN ese viaje pasa por los 128 estados en cadena, y en cada eslabón el gradiente queda multiplicado por un número (Bengio y otros, 1994).

En mi RNN recién creada ese número es de alrededor de 0,58. Parece poco, pero multiplicar 0,58 por sí mismo 127 veces da del orden de 10^{-30} . Es decir, al primer paso de la ventana no le llega prácticamente nada de la señal de error, y la red no puede aprender de lo que pasa ahí. Si el número fuera mayor que 1 pasaría lo contrario: 1,1 multiplicado 127 veces da más de 180.000, y el gradiente explota.

Por eso la RNN simple, en la práctica, aprende casi solo con el final de la ventana. El anexo A.3 muestra de dónde sale el 0,58 a partir de los gradientes que medí.

3.5 LSTM

La LSTM (Hochreiter y Schmidhuber, 1997) se diseñó para resolver justamente este problema. La idea es agregar una memoria aparte, que se puede pensar como un cuaderno de notas que viaja a lo largo de la secuencia. En cada paso, tres compuertas deciden qué hacer con ese cuaderno. Cada compuerta es un número entre 0 y 1 que la red calcula mirando el dato nuevo y el estado anterior:

- La compuerta de olvido decide cuánto del cuaderno conservar. Con 1 se conserva todo y con 0 se borra.
- La compuerta de entrada decide cuánto de la información nueva anotar.
- La compuerta de salida decide cuánto del cuaderno mostrar hacia afuera en ese paso.

La diferencia clave con la RNN está en cómo se actualiza la memoria. La RNN reescribe todo su estado en cada paso, pasándolo otra vez por la $\tanh$ y por los pesos U . La LSTM, en cambio, solo corrige su cuaderno: conserva una parte de lo que tenía y le suma lo nuevo. Si la compuerta de olvido vale cerca de 1, la memoria pasa casi intacta al paso siguiente, y el gradiente viaja hacia atrás por ese mismo camino sin achicarse.

Ejemplo: con la compuerta de olvido en 0,99, después de 100 pasos queda $0,99^{100} \approx 0,37$ de la información original, algo más de un tercio. Con el factor 0,58 de la RNN simple quedaría $0,58^{100} \approx 2 \times 10^{-24}$, nada. La LSTM no tiene la compuerta fija: aprende sola cuándo le conviene recordar (compuerta cerca de 1) y cuándo olvidar (cerca de 0).

Con un estado de 74 valores la LSTM tiene 25.610 parámetros, menos estado que la RNN simple (151) porque cada compuerta necesita sus propios pesos. En las pruebas también probé la GRU (Cho y otros, 2014), una versión simplificada con dos compuertas, pero dio el mismo resultado que la LSTM y la dejé afuera (sección 7).

3.6 Híbrido CNN + LSTM

Por último junté las dos ideas. Las dos primeras convoluciones y el MaxPool de la CNN detectan patrones locales y dejan una secuencia de 64 pasos, y una LSTM recorre esa secuencia de patrones en lugar de la señal cruda. La pregunta es si ordenar en el tiempo los patrones que encuentra la CNN agrega algo.



Figura 2. Arquitectura de los cuatro modelos. Debajo de cada capa se indica la forma de su salida (pasos $\times$ canales). Las flechas curvas marcan la recurrencia.

4. Entrenamiento y métricas

Entrené los cuatro modelos de la misma forma: optimizador Adam con tasa de aprendizaje 0,001, batches de 64 ventanas, dropout de 0,3 antes de la capa final y semilla fija 0 para que todo sea reproducible. El entrenamiento dura hasta 60 épocas con early stopping: si la pérdida de validación no mejora durante 15 épocas se corta, y me quedo con los pesos de la mejor época.

4.1 Función de pérdida: entropía cruzada

La red devuelve una probabilidad para cada clase, y la pérdida mira solo la que le dio a la clase correcta:

$$\text{pérdida} = -\ln(\text{probabilidad asignada a la clase correcta})$$

Ejemplos: si el modelo le da 90% a la clase correcta, la pérdida es $-\ln(0,9) = 0,11$. Si le da 50%, es 0,69. Si le da 10%, es 2,30. La pérdida castiga fuerte cuando el modelo se equivoca con confianza. Una pérdida promedio de 0,05 equivale, a grandes rasgos, a darle alrededor de 95% a la clase correcta.

Por eso uso la pérdida y no el accuracy para decidir cuándo cortar. El accuracy solo cambia cuando una predicción cambia de clase, mientras que la pérdida también mejora cuando el modelo gana confianza en las que ya acertaba.

4.2 Recorte de gradiente

Por la inestabilidad de las recurrentes apliqué recorte de gradiente a todos los modelos, para que la comparación sea pareja (Pascanu y otros, 2013). Si el tamaño (la norma) del gradiente supera 1, se lo achica hasta que valga 1 sin cambiar su dirección:

$$g_{\text{recortado}} = g \cdot \min(1, 1 / \|g\|)$$

Ejemplo: un gradiente (3, 4) tiene norma $\sqrt{3^2 + 4^2} = 5$. Se multiplica por 1/5 y sus componentes pasan a ser 0,6 y 0,8: apunta hacia el mismo lado pero con norma 1. Un gradiente con componentes 0,3 y 0,4 tiene norma 0,5 y no se toca. En la RNN simple llegué a ver normas de más de 200, que sin recorte habrían producido un paso 200 veces más grande de lo normal.

Funciona como un límite de velocidad. No ayuda contra el desvanecimiento, porque un gradiente chico no se agranda.

4.3 Métricas

El accuracy es el porcentaje de ventanas bien clasificadas, y sirve porque las clases están bastante balanceadas. El F1 macro calcula para cada clase una combinación de precisión y recall, y después promedia las seis clases con el mismo peso:

$$F1 = 2 \cdot \text{precisión} \cdot \text{recall} / (\text{precisión} + \text{recall})$$

Ejemplo: hay 100 ventanas reales de sentado. El modelo predice sentado 90 veces y acierta 80. La precisión es $80 / 90 = 0,89$ (de lo que dijo sentado, cuánto era sentado) y el recall es $80 / 100 = 0,80$ (de todo lo que era sentado, cuánto encontró). El F1 queda $2 \cdot 0,89 \cdot 0,80 / (0,89 + 0,80) = 0,84$. Promediar por clase evita que una clase mal resuelta quede escondida detrás de las fáciles.

La matriz de confusión completa la evaluación: muestra en qué clases se equivoca cada modelo y con cuáles las confunde.

5. Resultados

Tabla 1. Resultados en test (9 personas no vistas) y accuracy de validación en la época elegida.

Modelo	Parámetros	Mejor época	Validación	Test	F1 test
CNN 1D	25.302	25 de 40	98,9%	94,2%	0,942
RNN simple	25.374	29 de 44	97,3%	87,5%	0,876
LSTM	25.610	31 de 46	97,2%	91,3%	0,914
CNN + LSTM	25.620	22 de 37	98,9%	92,9%	0,930
Regresión logística	6.918			58,3%	0,554

Todas las redes superan con amplitud a la regresión logística (58,3%), lo que confirma que un modelo lineal no puede aprender de la señal cruda. Entre las redes, el orden es CNN 1D, CNN + LSTM, LSTM y RNN simple. La hipótesis inicial no se cumplió: la CNN, que no tiene memoria, fue la mejor.

También se ve que validación da entre 97% y 99% mientras test da entre 87% y 94%. No parece sobreajuste, porque la pérdida de validación queda incluso por debajo de la de train (en train el dropout está activo). Lo más probable es que las 5 personas de validación sean más fáciles que las 9 de test, algo que retomo en la sección 7.

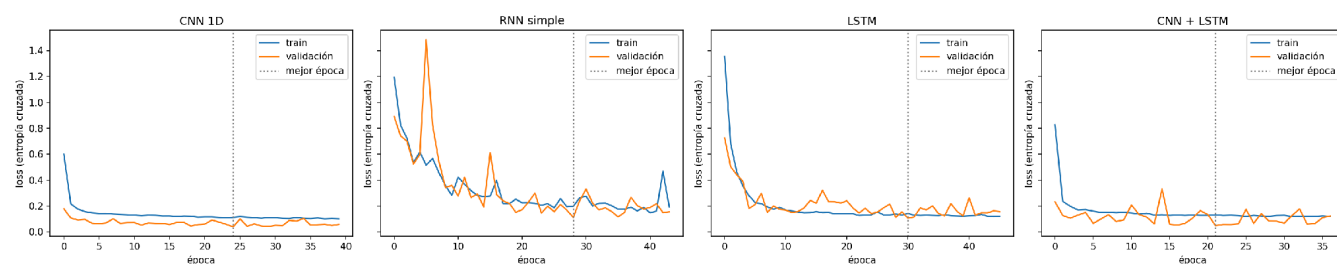


Figura 3. Pérdida de train y de validación por época. La línea punteada marca la época con menor pérdida de validación.

En la figura 3, la CNN y el híbrido bajan la pérdida en pocas épocas y quedan estables. La RNN simple es la más inestable, con saltos como el pico de 1,48 en la época 6, que corresponden al gradiente que explota aun con el recorte. La LSTM baja de forma más ordenada que la RNN simple. En todos los modelos la curva de validación salta más que la de train, porque se mide sobre pocas personas.

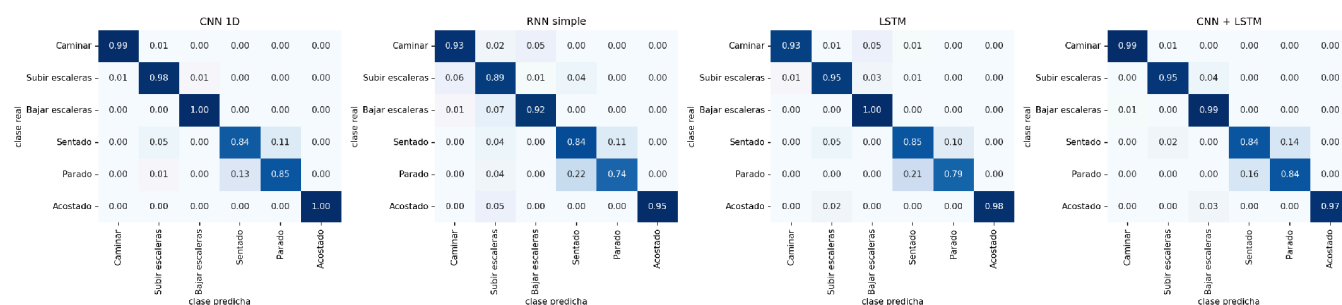


Figura 4. Matrices de confusión en test, normalizadas por clase real.

La figura 4 muestra que casi todo el error está en sentado contra parado. La CNN confunde el 11% de las ventanas de sentado con parado y el 13% de parado con sentado. En las dos posturas el celular queda casi con la misma orientación y no hay movimiento, así que la señal ofrece muy poco para distinguirlas. En las actividades dinámicas la CNN y el híbrido aciertan entre 95% y 100%.

6. Análisis

6.1 ¿Los modelos usan el orden temporal?

Tomé los modelos ya entrenados y les mostré el test desordenado. Corté cada ventana en bloques y mezclé el orden de los bloques, dejando intacta la señal dentro de cada uno. Con bloques de 128 pasos la ventana queda igual y con bloques de 1 se mezcla cada paso. La idea es como cortar un video en clips y pasarlos en otro orden: si con clips largos igual se reconoce la actividad, el modelo no depende del orden entre ellos.

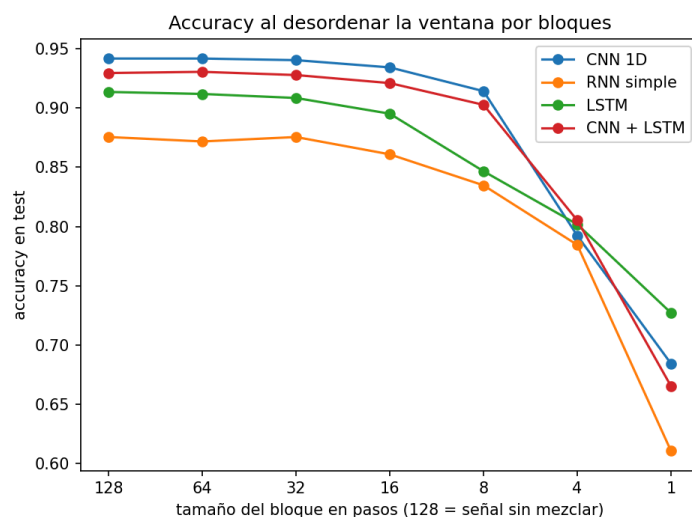


Figura 5. Accuracy en test al desordenar la ventana por bloques de distinto tamaño.

La CNN casi no cambia hasta bloques de 16 pasos (de 94,2% a 93,4%) y recién cae fuerte con bloques de 4. Tiene sentido, porque cada neurona mira 18 pasos: mientras el bloque sea más grande, el patrón queda entero. El híbrido se comporta casi igual. Las recurrentes, en cambio, empiezan a caer antes: la LSTM baja a 89,5% con bloques de 16 y a 84,7% con bloques de 8. O sea, las recurrentes sí usan el orden a mayor distancia, y aun así obtienen menos accuracy. En esta tarea lo que distingue las actividades es la forma local del movimiento, que la CNN captura sin necesitar el orden largo.

6.2 Cuánto influye cada paso de la ventana

Para ver el desvanecimiento del gradiente medí, para cada paso de la ventana, cuánto cambiaría la pérdida si ese paso cambiara un poco. Si el valor es casi cero, ese paso no influye y la red no puede aprender de él. La figura 6 lo muestra en escala logarítmica, donde cada línea de la grilla es 10.000 veces la anterior.

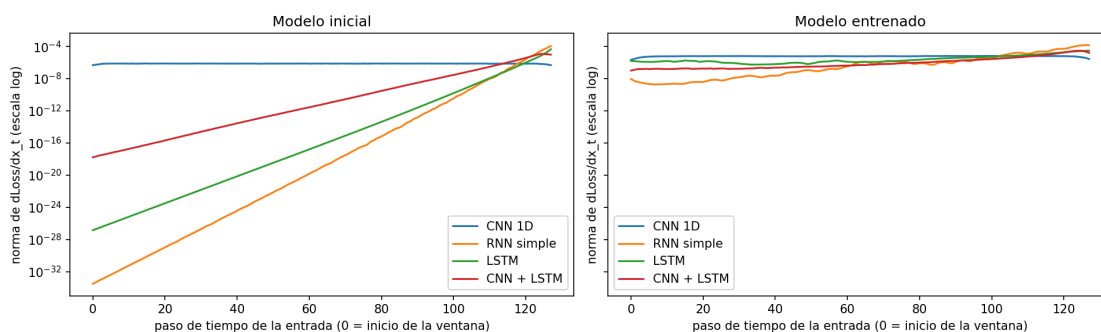


Figura 6. Influencia de cada paso de la entrada en la pérdida, con los modelos recién creados y ya entrenados.

Con los modelos recién creados, la CNN es plana: todos los pasos influyen lo mismo. Las recurrentes forman rectas, que es lo que se ve en escala logarítmica cuando algo se multiplica siempre por el mismo número. La RNN simple pierde 29,6 órdenes de magnitud entre el último y el primer paso, la LSTM 22,5 y el híbrido 12,8, porque su LSTM recorre 64 pasos y no 128. Después de entrenar, la LSTM queda casi plana: aprendió a mantener abierta su memoria. La RNN simple sigue unos 4,2 órdenes por debajo al principio de la ventana, o sea que los primeros pasos influyen unas diez mil veces menos que los últimos.

Esto explica por qué la LSTM mejora a la RNN simple (91,3% contra 87,5%), y también por qué no le gana a la CNN: poder recordar el principio de la ventana ayuda poco cuando la actividad se repite dentro de ella.

7. Lo que no funcionó y lo que aprendí

- La hipótesis inicial era falsa. Esperaba que la memoria de las recurrentes fuera una ventaja por tratarse de una serie temporal. Aprendí que lo importante no es que los datos sean una secuencia, sino qué tipo de estructura temporal define la clase. En HAR un paso dura cerca de medio segundo y se repite varias veces dentro de la ventana, así que recordar el principio no aporta. Otros trabajos también encontraron que las CNN igualan o superan a las recurrentes en muchas tareas de secuencias (Bai y otros, 2018).
- Los CSV de Kaggle no servían. Encontré el dataset en Kaggle en dos archivos CSV, pero traen solo las 561 features, sin eje de tiempo. Para una red secuencial necesitaba la señal cruda, que está únicamente en el repositorio de UCI.
- Features calculadas contra señal cruda. En una primera prueba, la regresión logística sobre las 561 features llegó a 95,3%, más que cualquier red, mientras que sobre la señal cruda obtuvo 58%. Entendí que así estaba comparando modelos con entradas distintas y decidí que todas las pruebas partieran de la señal cruda.
- La GRU no aportaba nada distinto. En corridas con tres semillas, GRU y LSTM promediaron 90,8% cada una. Me quedé con la LSTM, que explica de forma más directa el problema del gradiente, y usé ese lugar para el híbrido.
- La RNN simple fue inestable. Aparecieron picos en la pérdida y, en algunas épocas, normas de gradiente de más de 200. Eso me llevó a usar el recorte de gradiente.
- Validación poco representativa. Con 5 personas, validación dio entre 97% y 99% y test entre 87% y 94%. El early stopping y cualquier decisión tomada con validación dependen de esas pocas personas.
- Una sola semilla tiene ruido. En una prueba con tres semillas, la RNN simple varió entre 85,6% y 89,3%. Diferencias de uno o dos puntos entre modelos pueden deberse al azar, así que no saqué conclusiones de diferencias chicas.

8. Qué haría con más tiempo

- Probar con CAPTURE-24 (Chan y otros, 2024), que registra a 151 personas durante días enteros de su vida normal. En HAR cada ventana se clasifica sola. En un día real, en cambio, las actividades duran mucho y siguen un orden: alguien que durmió las últimas dos horas probablemente siga durmiendo. Una ventana de quietud, mirada sola, puede ser dormir, mirar

televisión o trabajar sentado, y lo que pasó en las ventanas anteriores ayuda a distinguirlas. Los modelos del paper mejoran justamente cuando tienen en cuenta varias ventanas seguidas. Ahí la memoria de una recurrente trabajaría a escala de minutos u horas y no dentro de 2,56 segundos, y es el escenario donde esperaríamos ver la ventaja que no apareció en HAR.

- Validación cruzada por persona, rotando qué personas quedan en validación, para que las decisiones no dependan de 5 personas.
- Varias semillas para cada modelo, para informar un promedio y cuánto varía en lugar de un solo número.
- Entrenar cada arquitectura con distintos tamaños para ver dónde deja de mejorar cada una y si el orden entre ellas se mantiene.
- Otras variantes, como la LSTM bidireccional o capas recurrentes apiladas, y arquitecturas más recientes como los Transformers.

9. Conclusiones

En el reconocimiento de actividades con UCI HAR, la memoria de las redes recurrentes no fue una ventaja. La CNN 1D, cuyas neuronas miran 0,36 segundos cada una, obtuvo el mejor resultado (94,2%), por encima del híbrido CNN + LSTM (92,9%), la LSTM (91,3%) y la RNN simple (87,5%). Los dos experimentos explican por qué. La mezcla por bloques muestra que las recurrentes usan el orden largo, pero eso no mejora la clasificación. La influencia por paso muestra que la LSTM corrige el desvanecimiento del gradiente de la RNN simple, pero recordar el principio de la ventana aporta poco cuando la actividad se repite dentro de ella.

La conclusión general es que elegir entre convolución y recurrencia depende de la estructura temporal del problema. Cuando la información es local y se repite, la convolución alcanza y es más fácil de entrenar. La recurrencia tiene sentido cuando hace falta recordar algo lejano. El error que queda, sentado contra parado, no depende del tiempo sino de la orientación del celular, y ninguna de las arquitecturas lo resolvió.

Referencias

- Anguita, D., Ghio, A., Oneto, L., Parra, X. y Reyes-Ortiz, J. L. (2013). A Public Domain Dataset for Human Activity Recognition Using Smartphones. ESANN 2013.
- Bai, S., Kolter, J. Z. y Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271.
- Bengio, Y., Simard, P. y Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 5(2).
- Chan, S. y otros (2024). CAPTURE-24: A large dataset of wrist-worn activity tracker data collected in the wild for human activity recognition. Scientific Data, 11.
- Cho, K. y otros (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. EMNLP 2014.
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press, capítulos 9 y 10.
- Hochreiter, S. y Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8).
- Pascanu, R., Mikolov, T. y Bengio, Y. (2013). On the difficulty of training recurrent neural networks. ICML 2013.
- Dataset: <https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Anexo A. Ayuda visual de la matemática detrás de las CNN

Las figuras de este anexo las generé con una página interactiva que armé para estudiar estos temas, usando los mismos valores que tiene mi CNN. La página permite cambiar el kernel, el padding y la cantidad de capas y ver cómo cambian las cajas: <https://nicoferrero.github.io/tp-redes-neuronales-deep-learning/>

A.1 Padding: por qué la señal no se acorta

Un filtro de 5 cajas no puede centrarse en la primera ni en la última caja de la señal, porque se saldría. Sin padding, cada convolución pierde 4 pasos (128 pasa a 124). Agregando 2 ceros de cada lado, el filtro entra en todas las posiciones y la salida conserva los 128 pasos. La regla es agregar $(k - 1) / 2$ ceros de cada lado, que con un filtro de 5 da 2.



Figura 7. Ejemplo con 10 pasos, filtro de 5 y padding 2. Cada salida es el promedio de las 5 cajas que cubre el filtro. La salida mide lo mismo que la entrada.

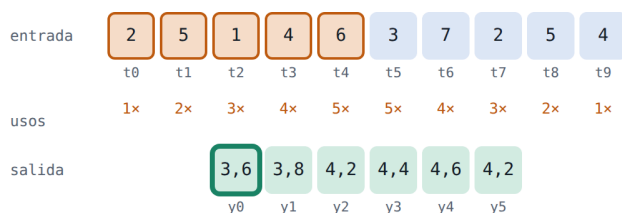


Figura 8. La misma señal sin padding. La salida se acorta 4 pasos, y la fila de usos muestra que los bordes se miran menos veces que el centro.

A.2 Campo receptivo: cuánto ve cada neurona

Cada neurona de salida mira solo una parte de la entrada, y esa parte tiene forma de cono: se ensancha a medida que se baja hacia la entrada. Hay un cono por cada posición de salida, todos con la misma forma y corridos un paso entre sí, porque el filtro usa los mismos pesos en todas las posiciones. Ese ancho en la entrada es el campo receptivo.

En la figura 9 conviven dos cosas. Las cajas azules son las capas completas: cada fila tiene la misma cantidad de cajas de punta a punta, porque el padding evita que las convoluciones acorten la señal, y las dos filas de arriba tienen cajas del doble de ancho porque el MaxPool redujo el largo a la mitad. Las cajas naranjas son el cono de una sola neurona, la verde. El padding cambia el largo de la capa, pero no cuántas cajas mira cada neurona, y por eso el cono se achica hacia arriba aunque la capa no se achique.

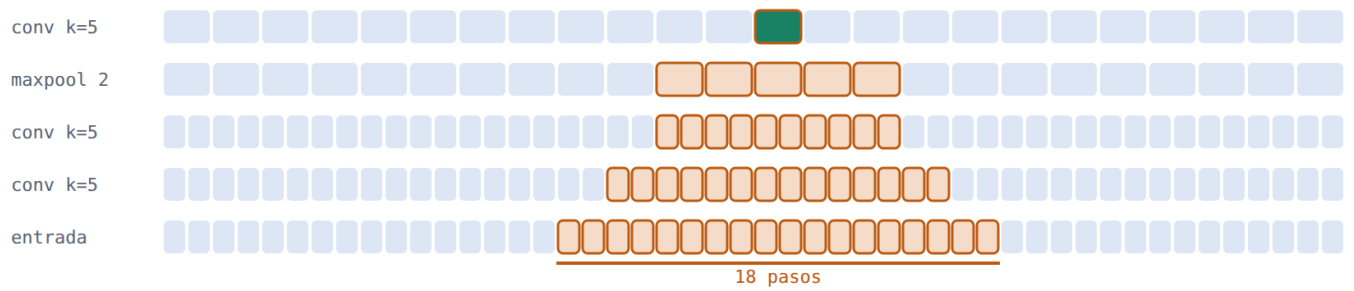


Figura 9. La neurona central de la última capa (en verde) y todas las cajas que influyen en ella. De arriba hacia abajo el cono tiene 1, 5, 10, 14 y 18 cajas. Se muestran 48 pasos de la señal para que entren en la página.

El cono crece 4 pasos por capa, y no se multiplica por 5, porque las neuronas vecinas miran casi los mismos pasos. En la figura 10, una neurona de la conv 2 mira 5 neuronas de la conv 1 y cada una mira 5 pasos: son 25 cajas, pero solo 9 pasos distintos, porque únicamente los extremos agregan lugares nuevos. Después del MaxPool cada caja vale 2 pasos, así que la última capa suma el doble y el cono llega a 18.

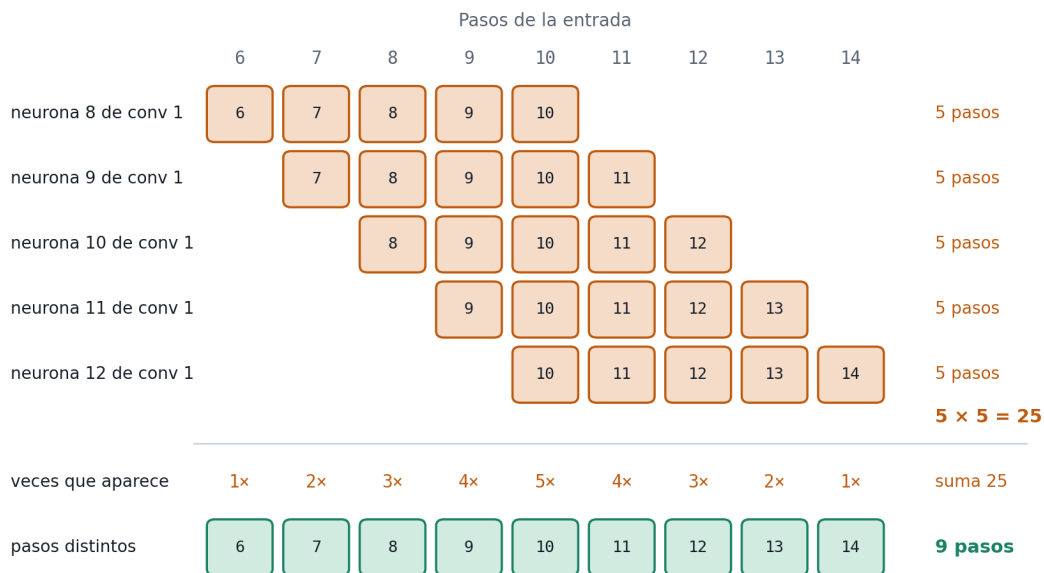


Figura 10. Las 5 neuronas de la conv 1 que mira una neurona de la conv 2. Contando todas las cajas hay 25, pero los pasos distintos son 9.

A.3 De dónde sale el 0,58

En la RNN recién creada medí el gradiente en el último paso y en el primero. La cuenta para obtener el número por el que se multiplica en cada paso tiene tres partes:

- Cuántas veces hay que dividir por 10 para ir de uno al otro: $\log_{10}(1,15 \times 10^{-4} \div 2,99 \times 10^{-34}) = 29,6$ órdenes de magnitud.
- Repartido en 127 pasos: $29,6 / 127 = 0,233$ órdenes por paso.
- Convertido a un factor: $10^{-0,233} \approx 0,58$.

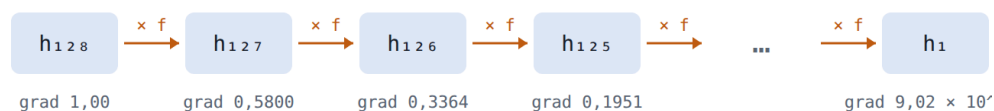


Figura 11. Cadena de estados de la RNN. Cada flecha hacia atrás multiplica el gradiente por 0,58.

Tabla 2. Factor por paso calculado a partir de los gradientes medidos. Cuanto más cerca de 1, mejor pasa el gradiente.

Modelo	Órdenes perdidos	Pasos	Factor por paso
RNN simple, recién creada	29,6	127	0,58
LSTM, recién creada	22,5	127	0,66
CNN + LSTM, recién creada	12,8	64	0,63
RNN simple, entrenada	4,2	127	0,93
LSTM, entrenada	1,3	127	0,98